

Backend + ML/AI Engineer Roadmap

Goal: Backend engineer + ML engineer with strong AI/app implementation. CCC Senior top 10%, aim for CCO. **Starting point:** CS50 ✓, CS50P (in progress), CCC ~J5/S2, a few games, a few GPT wrappers. **Stack:** Python (FastAPI), PostgreSQL, Redis, React (TypeScript), Docker/GitHub Actions, PyTorch, standard ML/AI Python libs. **Pace:** ~8-12 hrs/week during school, compress in summers. ~2.5-3 years total (Gr. 10 → Gr. 12+).

How-to-learn shorthand used in tables

Label	What it means
Do-first	Attempt the problem/project cold before reading or watching anything. Struggle is the point.
Type-along → variant	Follow the tutorial by typing (not pasting), then immediately close it and build a small variant from memory.
Read → implement	Read one section/chapter, close it, implement the core idea from scratch, then check. Never read two sections back-to-back without implementing between them.
Watch → code same day	Watch the video, then write code applying the concept before the day ends. Watching alone fades within 48 hrs.
Timed reps	Do it under real time pressure with no hints open. Review only after a genuine attempt.
Spaced re-solve	After understanding a solution, re-solve the same problem cold ~1 week later without notes. Builds durable memory.

Label	What it means
AI-assisted	Use an AI to get unstuck, check your reasoning, or explore "what if" variants — but write the code yourself first. Never have AI produce the first draft.
Read-only	Reading/watching is sufficient — this is reference or orientation material, not a skill to produce.

Effort Allocation

Phase	Area	%	Approx. Duration
0	DSA & Competitive Programming	5%	Ongoing — 1 session/week forever
1	Programming Fundamentals	7%	4–5 weeks
2	Backend Engineering	23%	12–14 weeks
3	System Design	5%	4 weeks
4	DevOps & Cloud	7%	4–5 weeks
5	Math & Stats for ML	9%	7–9 weeks, paced gently
6	Classical ML	9%	6–7 weeks
7	Deep Learning	9%	6–7 weeks
8	Applied AI / LLMs / MLOps	13%	8–9 weeks
9	Capstone Projects	10%	9–11 weeks (overlaps)
—	Side fields	~3%	Ongoing

Ratio breakdown: Backend + infra (Phases 0–4) is foundational infrastructure — not competing with AI work, just the floor that makes everything shippable. The 80:20 split applies within the side fields: ~80% of that budget goes to AI-adjacent skills (data engineering, security, open source contributing, API design) and ~20% to pure app-layer skills (frontend, mobile, technical writing).

Phase 0 — DSA & Competitive Programming (ongoing, 1 session/week)

Target: CCC Senior top 10% (reliable S1–S4, partial S5 $\approx$ USACO Gold). CCO is possible with consistent weekly practice over 2+ years — not guaranteed, but not unrealistic.

Each session is one of:

- Timed CCC past paper under full contest conditions, OR
- One new Gold-level topic + 2–3 problems on it

Alternate between the two — timed practice is what moves the needle, not topic study alone.

Language for contests: Switch to C++ for all contest practice once you've finished the C++ section in Phase 1. Python is too slow for CCC Senior time limits at Gold-level problem sizes — you'll hit TLE on problems your algorithm is actually solving correctly. The switch is worth doing early rather than relearning everything later under time pressure.

Topic	Resource	How to learn
Quick Silver review (two pointers, sliding window, hashing, basic graphs)	USACO Guide — Silver track	Skim fast. For anything not already solid: timed reps on 1-2 problems, then spaced re-solve .
Graphs (Dijkstra, MST, topological sort)	USACO Guide — Gold graphs	Read → implement each algorithm from scratch before touching problems. Then timed reps on Gold problems. AI-assisted to check your Dijkstra implementation's edge cases.
DP (tree DP, bitmask DP, interval DP)	USACO Guide — Gold DP; <i>Competitive Programmer's Handbook</i> (free PDF)	Do-first on each problem — minimum 30 min of genuine attempt before the editorial. Editorial: read, close, spaced re-solve a week later.
Greedy (exchange argument, common patterns)	USACO Guide — Gold greedy	Read → implement the exchange argument proof for each pattern, then timed reps .
Range queries (segment trees, Fenwick trees)	USACO Guide — Gold "Range Queries"	Read → implement from scratch (no copy-paste). Build a Fenwick tree by hand before looking at segment trees. AI-assisted to debug index-off-by-one issues only.
Number theory (modular arithmetic, GCD, primes, combinatorics)	<i>Competitive Programmer's Handbook</i> — number theory chapters	Read → implement each primitive (fast exp, sieve, modular inverse). Then apply in 1-2 contest problems.

Topic	Resource	How to learn
Reference	cp-algorithms.com	Read-only — look up when you hit an unfamiliar algorithm, not as a reading list.
Contest practice	CEMC official CCC/CCO past papers + editorials (cemc.uwaterloo.ca)	Timed reps — full contest conditions every other week. After: read editorial only for problems you couldn't solve, then spaced re-solve .
Online judge	DMOJ CCC archive	Timed reps + spaced re-solve for any problem that required an editorial.

Skip LeetCode for now. Pick up NeetCode 150 a few weeks before your first internship application — Gold-level material already covers the substance.

Phase 1 — Programming Fundamentals (7%)

Do C++ first, before or alongside finishing CS50P. Everything else in this phase is Python and can follow after.

Topic	Resource	How to learn
C++ for competitive programming (syntax, STL — vectors, maps, sets, priority queues, pairs; I/O with <code>cin</code> / <code>cout</code> ; pointers and references; basic memory model)	<i>Competitive Programmer's Handbook</i> ch. 1–2 for contest-specific idioms; learncpp.com for language fundamentals	Type-along → variant through learncpp.com for the core language (do this linearly — C++ has real footguns that trip up Python programmers, especially around references and undefined behaviour). After the basics are solid, switch to timed reps on DMOJ with C++ immediately — the only way to get comfortable with contest I/O and STL under pressure is to actually use them in contest problems. AI-assisted to understand compiler errors, which are famously cryptic in C++. Target: able to solve Silver-level problems fluently in C++ within 3–4 weeks of starting.
Finish CS50P	CS50P (in progress)	Do-first on every problem set. Attempt cold, get stuck, push through — then check. Don't touch staff solutions until you've submitted something of your own.
Intermediate Python (comprehensions, generators, decorators, context managers, type hints, venv/poetry)	Real Python tutorials; Corey Schafer's Python playlist	Type-along → variant for each feature. After the decorator tutorial, close it and write your own timing decorator from scratch before moving on.

Topic	Resource	How to learn
OOP in depth (inheritance vs. composition, basic SOLID)	Corey Schafer's "Python OOP" series	Type-along → variant — after each video, rebuild the example class from memory, then write a second one for a different domain (e.g., if the tutorial uses <code>Dog</code> , you build <code>BankAccount</code>). AI-assisted to check if your composition vs. inheritance choice makes sense.
Git (branching, merging, conflict resolution, PR workflow)	<i>Pro Git</i> (free, git-scm.com/book)	Read → implement — read a chapter, then actually do it in a throwaway repo. Deliberately create a merge conflict and resolve it. Reading about Git without doing it in a terminal doesn't stick.
CLI & Linux (filesystem, permissions, piping, bash basics, SSH)	<u>MIT Missing Semester</u> (free)	Do-first on every exercise before checking solutions. Use your actual terminal throughout — don't read about commands you never run.
Testing (pytest, basic TDD, mocking)	<i>Test-Driven Development with Python</i> — Percival (free, obeythetestinggoat.com)	Type-along → variant — write the test before the implementation on every exercise, even when it feels backwards. The discipline is the point.

Phase 2 — Backend Engineering (23%)

Start with CS50W. It covers Flask, Django, SQL, JS, APIs, and security in one course

with real project reps. Layer FastAPI on top afterward — that's what you'll actually use for ML-serving in Phase 8.

Topic	Resource	How to learn
Full-stack foundations	CS50W	Do-first on every project. Finish the project yourself before looking at any walkthrough. These are the reps that matter most in this phase.
HTTP (methods, status codes, headers, statelessness)	MDN HTTP docs	Read-only + one experiment: use <code>curl</code> to manually send a GET, POST, and bad request to a live API and read the raw responses.
FastAPI (routing, Pydantic, dependency injection, async basics)	FastAPI official tutorial	Type-along → variant — after each tutorial section, close it and build the same endpoint for a different resource from memory. AI-assisted to understand Pydantic validation errors when they're cryptic.
PostgreSQL deep dive (joins, indexes, transactions, normalization, schema design)	SQLBolt → pgexercises.com → PostgreSQL official tutorial	Timed reps on SQLBolt and pgexercises. For schema design: do-first on design problems, then compare against normal form rules.
SQLAlchemy ORM (models, migrations, queries)	SQLAlchemy official docs	Type-along → variant — build a small model set, then rebuild it for a different domain from memory. AI-assisted for Alembic migration errors.
Redis (caching, sessions, pub/sub basics)	Redis official "Introduction to Redis"	Type-along → variant — implement a cache-aside pattern manually in Python after the intro, without looking

Topic	Resource	How to learn
		at examples. One weekend covers what you need.
Auth (bcrypt, JWT, OAuth2 flow)	FastAPI security docs	Type-along → variant — implement the full login + protected route flow once, then rebuild it from scratch in a blank project. AI-assisted to reason through the OAuth2 flow diagram, not to generate the code.
Background tasks & queuing (Celery + Redis)	FastAPI docs + Celery official quickstart	Type-along → variant — get one background task working end-to-end, then add a second task of your own design.
API testing (pytest + httpx/TestClient)	FastAPI testing docs	Do-first — write tests before endpoints on at least one project.
Practice reps	Build 3-4 small APIs; second-to-last one should call an LLM endpoint to bridge into Phase 8	Do-first on all specs. AI-assisted only to get unstuck on specific errors, never to scaffold.

Phase 3 — System Design (5%)

Topic	Resource	How to learn
Core concepts (load balancing, caching,	<u>System Design Primer</u> (free)	Read → implement in the sense of: read a concept, close it, then write a half-page

Topic	Resource	How to learn
CAP theorem, replication)		explanation of it in your own words. If you can't, you don't know it yet.
Scaling patterns (sharding, message queues, CDNs)	ByteByteGo YouTube	Watch → code same day doesn't apply here — instead: watch, then draw the architecture from memory on paper before the next video. No drawing = no retention.
Deep dive on hard problems	<i>Designing Data-Intensive Applications</i> — Kleppmann	Read → implement (as above — explain in your own words after each chapter). This is a dense book; one chapter per sitting is the right pace.
Practice (design 3–4 classic systems on paper)	<i>System Design Interview</i> — Alex Xu Vol. 1; URL shortener, rate limiter, chat app, ML feature store	Do-first — attempt a full design on paper before reading Xu's chapter on it. AI-assisted to stress-test your design ("what breaks at 10M users?") after you've done it yourself.

Phase 4 — DevOps & Cloud (7%)

Topic	Resource	How to learn
Docker (images, Dockerfile, docker-compose)	Docker official "Get Started"	Type-along → variant — containerize one of your own Phase 2 APIs after the tutorial, without referring back to it. Deliberately break something (delete a layer, mess up the port) and debug it.

Topic	Resource	How to learn
CI/CD pipelines	GitHub Actions official docs	Type-along → variant — get a pipeline running for a real project, then add a step (linting, test coverage threshold) you designed yourself. AI-assisted to decode YAML errors.
First real deployment	Render.com or Railway	Do-first — try to deploy without a tutorial first. Read the docs only when you hit an actual error. Shipping something broken and fixing it teaches more than a clean walkthrough.
Cloud fundamentals	AWS Skill Builder — "Cloud Practitioner Essentials" (free)	Read-only for now — orientation, not skill-building. You'll do real AWS work in Phase 8.
Monitoring (logs, metrics, alerts)	freeCodeCamp Docker/K8s crash course (YouTube)	Watch → code same day — after the logging section, add structured logging to one of your existing APIs that day.

Phase 5 — Math & Stats for ML (9%)

Going into Gr. 11 — some of this is genuinely ahead of school. Let school carry algebra/trig; focus extra effort on the ML-specific slices it won't cover.

Topic	Notes	Resource	How to learn	Timing
Algebra & trig fluency	Your Gr. 11 course itself	Khan Academy Algebra 2 / Precalculus	Use Khan Academy only when a school topic feels shaky — timed reps on the exercises for that specific topic, not as a separate study track.	Ongoing through school
Linear algebra (vectors, matrices, matrix mult., eigenvectors)	No calculus needed — safe to start now	3Blue1Brown "Essence of Linear Algebra"; Khan Academy Linear Algebra for practice	Watch → code same day for 3B1B: after each video, implement the operation in NumPy (e.g., matrix multiply by hand, then verify with <code>np.dot</code>). Then timed reps on Khan Academy exercises for the same topic.	Start now
Probability & stats (distributions, Bayes, expectation/variance)	No calculus needed — builds on algebra	StatQuest (YouTube); Khan Academy Stats & Probability	Watch → code same day for StatQuest: after each video, simulate the concept in Python (e.g., sample from a distribution, verify Bayes' theorem numerically). Timed reps on Khan Academy exercises.	Start now

Topic	Notes	Resource	How to learn	Timing
Calculus (derivatives, gradients, chain rule only)	Genuine Gr. 12 material — go slowly	Khan Academy "Differential Calculus" — derivative + chain rule units only	Read-only + exercises at a slow pace. Work every Khan Academy exercise in the unit — don't skip the practice problems. No rush; this is new ground.	After Gr. 11 math clicks
Optional: full formal treatment	Only after Gr. 12 calc	<i>Mathematics for Machine Learning</i> (free, mml-book.github.io)	Read → implement — work the exercises, don't just read the derivations.	Gr. 12 and beyond

Don't let this phase block Phase 6. Linear algebra and stats are enough to start — learn calculus just-in-time as backprop comes up in Phase 7.

Phase 6 — Classical ML (9%)

Topic	Resource	How to learn
Survey foundation (search, classical ML, intro neural nets)	CS50 AI	Do-first on every project. The projects are small but the constraint of implementing things yourself (minimax, naive Bayes, etc.) is where the learning is.
Python ML stack (NumPy, Pandas, Matplotlib)	Kaggle Learn micro-courses (free, fast)	Type-along → variant — after each notebook, close it and reproduce the key operations on a different dataset from memory. AI-assisted to decode Pandas

Topic	Resource	How to learn
		error messages quickly so you don't lose momentum.
ML workflow (train/test split, cross-validation, metrics)	Kaggle Learn "Intro to ML" + "Intermediate ML"	Type-along → variant — implement a full train/eval/test split pipeline from scratch after the tutorial, not during it.
Supervised learning (linear/logistic regression, decision trees, random forests, gradient boosting with XGBoost)	<i>Hands-On ML</i> — Géron, Part 1	Read → implement — one algorithm per session: read the chapter, implement it with scikit-learn from scratch (no copy-paste), then compare your results to the book's. AI-assisted to understand why your accuracy differs, not to fix the code.
Unsupervised learning (k-means, PCA)	Same book, relevant chapters	Same as above — implement, then apply to a dataset of your own choosing.
Optional lecture-style	Andrew Ng's "Machine Learning Specialization" (Coursera)	Watch → code same day if you use it — every lecture has accompanying assignments; do them before moving to the next lecture.
Kaggle practice	Titanic → tabular playground competition	Do-first — submit a baseline (raw data, single model) before reading any public notebooks. Then read one top notebook, understand it, and incorporate one idea at a time. Don't copy whole pipelines.

Phase 7 — Deep Learning (9%)

Complete the derivative/chain rule slice from Phase 5 before hitting backprop here.

Topic	Resource	How to learn
Neural net fundamentals (forward/backward pass, backprop intuition)	3Blue1Brown neural network series; CS50 AI neural nets week	Watch → code same day — after 3B1B's backprop video, implement a 2-layer net forward and backward pass in pure NumPy (no PyTorch). This is the single highest-leverage exercise in this phase.
PyTorch (tensors, autograd, <code>nn.Module</code> , training loops)	PyTorch official "60 Minute Blitz" + tutorials	Type-along → variant — after the Blitz, rebuild the same network for a different task (different dataset, different architecture) from memory. AI-assisted to interpret tensor shape errors — these are common and cryptic.
Practical top-down approach	fast.ai "Practical Deep Learning for Coders" (free)	Do-first on every notebook — run it, break something intentionally, fix it. Fast.ai's philosophy is top-down by design; trust the approach and build before you fully understand.
CNNs / RNNs	fast.ai, relevant lessons	Same as above — implement the architecture on a dataset you chose, not the one fast.ai uses.
Transformers (attention mechanism)	3Blue1Brown "Attention" video; d2l.ai transformer chapter	Watch → code same day for 3B1B. Then read → implement the scaled dot-product attention in PyTorch from the d2l chapter before looking at <code>nn.MultiheadAttention</code> . AI-assisted to walk through the matrix dimensions until the shapes make sense.
Optional textbook	<i>Dive into Deep Learning</i> (free, d2l.ai)	Read → implement — every chapter has runnable notebooks; run them, then modify. Don't read the chapter without running the code.

Phase 8 — Applied AI / LLMs / MLOps (20%)

You already know what an API call to a model looks like — this phase goes past that. This is the heaviest phase by allocation and the one most directly tied to your goal. Treat it accordingly.

Topic	Resource	How to learn
Prompt engineering properly	Anthropic prompt engineering docs (docs.claude.com)	Read → implement — after each technique (chain-of-thought, few-shot, XML tags), immediately test it against a task you care about. Keep a log of what worked and what didn't. AI-assisted (meta: use Claude to stress-test your own prompts).
Structured outputs & function calling	Anthropic + OpenAI function calling docs; Instructor library (Python)	Type-along → variant — get a model returning structured JSON reliably, then design your own schema for a different use case. This is the primitive that makes agents and pipelines composable.
Embeddings & vector DBs (pgvector, Chroma)	Hugging Face NLP Course (free)	Type-along → variant — embed a small document set, store in pgvector, run similarity queries. Swap in a different embedding model and compare results. AI-assisted to understand cosine similarity output when results look wrong.
RAG pipelines (end-to-end)	DeepLearning.AI: "LangChain for LLM Application Development," "Building and Evaluating Advanced RAG"	Type-along → variant — get the tutorial RAG pipeline working, then immediately rebuild it for a domain you chose. AI-assisted to diagnose retrieval failures (wrong chunks, bad similarity scores).

Topic	Resource	How to learn
RAG evaluation (recall, precision, faithfulness, answer relevance)	RAGAS library docs; DeepLearning.AI "Building and Evaluating Advanced RAG"	Do-first — try to define what "good retrieval" means for your own RAG app before reading the metrics. Then instrument your pipeline with RAGAS and interpret the numbers. Knowing how to eval separates junior from senior AI engineering.
Agents & tool use (multi-step LLM workflows, ReAct pattern)	DeepLearning.AI "Building Agentic RAG"; LangChain / LlamaIndex docs	Do-first after the tutorial — design and build an agent that solves a task you defined. AI-assisted to debug tool-calling logic and trace where the agent goes wrong.
Agentic frameworks in depth (LangGraph or AutoGen)	LangGraph docs + tutorials	Type-along → variant — build the tutorial graph, then design a second one with a loop and a human-in-the-loop step. Stateful multi-agent systems are where most production agentic work happens.
Observability & tracing for LLM apps (LangSmith or Langfuse)	LangSmith docs or Langfuse (open-source)	Type-along → variant — instrument one of your pipelines with tracing. You can't debug what you can't observe. This is the MLOps equivalent of logging.
Lightweight fine-tuning (LoRA/PEFT)	Hugging Face NLP Course — fine-tuning chapter	Type-along → variant — fine-tune on the tutorial dataset, then fine-tune the same base model on a small dataset of your own. Observe what changes.

Topic	Resource	How to learn
MLOps (model serving, versioning, monitoring drift)	Made With ML — Goku Mohandas (free)	Type-along → variant — follow the serving tutorial, then add a drift-detection check of your own design. Always pair each section with a real implementation step.
Cloud AI infra (GPU instances, managed serving, hosted vector DBs)	AWS SageMaker or GCP Vertex AI free-tier — pick one, don't split	Type-along → variant — deploy a model end-to-end, then modify the serving config. AI-assisted to parse IAM/permission errors.
Multimodal inputs (vision + text, document parsing)	Anthropic vision docs; LlamaParse or Unstructured for document ingestion	Type-along → variant — build one pipeline that takes an image or PDF as input and produces a structured output. Multimodal is increasingly table stakes for applied AI work.
Capstone tie-in	Rebuild one GPT wrapper as a proper RAG app with FastAPI backend, auth, tracing, eval, and deployment	Do-first — spec out the rebuild yourself before looking at any templates. Ship it with RAGAS eval scores you can actually report.

Phase 9 — Capstone Projects (10%)

3 deployed, end-to-end projects. Finish over scope.

Project	Stack	AI:app ratio	How to learn
RAG Q&A app	FastAPI + PostgreSQL + pgvector, React (TypeScript) frontend, Render/AWS, GitHub Actions CI/CD, RAGAS eval, LangSmith tracing	~85:15	Do-first on the full design before writing code. Ship v1 rough, add eval and tracing before calling it done — those two things are what make it portfolio-worthy vs. tutorial-level.
Agentic assistant	LangGraph stateful agent + tool-calling + external APIs (e.g. web search, calendar, code exec) + FastAPI serving layer + structured logging + eval harness	~90:10	Do-first on agent design (what tools, what loop, what failure modes, what the eval looks like) before any code. AI-assisted to design the eval harness and stress-test failure modes. This is the hardest project — do it last.
Classical ML API	scikit-learn or XGBoost model + FastAPI serving, pytest, basic drift monitoring, Docker	~60:40	Do-first — train and evaluate the model before building the API. This is the most backend-heavy project by design; it rounds out the portfolio rather than being the centrepiece.

Side Fields (~3%, each <1%)

80% of this budget goes to AI-adjacent skills; 20% to app-layer. Do these as interest and spare time allow — none are blockers.

Field	Resource	How to learn	Project / Outcome
App Security	PortSwigger Web Security Academy (free)	Do-first on every lab — attempt to find and exploit before reading the hint. Non-negotiable for this domain.	Find and fix one real vuln in your own API
Data Engineering	DataTalksClub "Data Engineering Zoomcamp" (free)	Type-along → variant — get the tutorial DAG running, then build one for a data source you chose	Airflow pipeline: raw data source → cleaned Postgres table feeding an ML model
Networking basics	<i>Computer Networking: A Top-Down Approach</i> — Kurose & Ross, first 3 chapters only	Read-only for orientation; watch → code same day for anything involving sockets	Understand what actually happens when your RAG app makes an API call; useful for debugging latency
Frontend (React + TypeScript)	Scrimba "Learn React" (free); TypeScript official docs	Type-along → variant — build the tutorial component, then rebuild from memory for a different use case. TypeScript: let compiler errors teach you after the basics.	Dashboard UI for one capstone API

Stack Summary

Layer	Choice	Why
Language	Python	Already know it; the ML/AI ecosystem is Python-native
API framework	FastAPI	Async, typed, OpenAPI-native, production-grade, ML-serving ready
Database	PostgreSQL + pgvector	Relational default + vector search without a separate DB
Caching / queuing	Redis + Celery	Standard backend pattern; needed for ML pipelines too
ORM	SQLAlchemy	Pairs with FastAPI cleanly, mature ecosystem
Frontend	React + TypeScript	Typed, component-based, industry standard
ML/DL	scikit-learn + PyTorch	Sklearn for classical, PyTorch for everything deep
LLM / AI libs	LangChain, LlamaIndex, Hugging Face	Standard applied AI stack
Containerisation	Docker + docker-compose	Local dev parity with production
CI/CD	GitHub Actions	Lightweight, free, integrates with everything
Deployment	Render / Railway → AWS	Low friction first, scale up later

Layer	Choice	Why
Cloud AI	AWS SageMaker or GCP Vertex (pick one)	Managed training + serving for Phase 8

Pacing

- Finish CS50P first.
- Phase 0 (DSA) is a weekly habit layered on top of everything else — never a phase you "finish."
- Start CS50W at the top of Phase 2; finish it, then move to FastAPI.
- Phases 4 and 5 can run in parallel with evening slots while deep in Phase 2 or 6.
- Start CS50 AI at the beginning of Phase 6; let it carry you into Phase 7.
- Don't let Phase 5 (Math) block Phase 6 — learn just-in-time as concepts appear.
- If Phase 8 pulls you in early given your wrapper background, let it; loop back for gaps.

Period	Focus
Gr. 10–11	Phases 0–2: DSA habit, fundamentals, backend; start linear algebra + stats when comfortable
Gr. 11–12	Phases 3–6: DevOps, System Design, calculus slice, Classical ML
Gr. 12+	Phases 7–9: Deep Learning, Applied AI, Capstone